JavaScriptの「コンストラクター関数」って何？

オブジェクトを量産するための「設計図」です！
たとえば「人」というオブジェクトを何人も作りたいときに便利です。
イメージ：
設計図（constructor） → 製品（オブジェクト）
function Human(...) → const hito = new Human(...)
「同じ型のオブジェクトを何回も作るなら、設計図を使った方が楽だよ！」

書き方と使い方（基本形） ステップ①：関数名を大文字でスタート

```
[ ]  function Human(name, age) {
       this.name = name;
       this.age = age;
     }
```

this は「作られるオブジェクト自身」を表す。

ステップ②：new を使って実体（オブジェクト）を作る

```
[ ]  const taro = new Human("太郎", 30);
     const hanako = new Human("花子", 25);
```

ステップ③：プロパティを呼び出す

```
[ ]  console.log(taro.name);  // 太郎
     console.log(hanako.age); // 25
```

どうして new を使うの？

new をつけることで "Human関数が特別な関数（コンストラクター）" として動きます！

1. 空のオブジェクト {} を作ってくれる！自動的に

```
const obj = {}; // ← これをnewがやってくれる
```

2.this を その新しいオブジェクトに紐づけてくれる！

```
function Person(name) {
  this.name = name; // ← thisが新しいオブジェクトを指すようになる
}
const p = new Person("たろう");
console.log(p.name); // "たろう"
```

3.returnしなくても、自動でそのオブジェクトを返してくれる！

```
function Person(name) {
  this.name = name;
  // return はいらない！
}
const p = new Person("はなこ");
console.log(p); // Person { name: "はなこ" }
```

同じパターンでたくさん作れる！

```
const minjal = new Human("ンジビカ", 28);
const minjal = new Human("カグマル", 33);

console.log(minjal.name); // ンジビカ
console.log(minjal.name); // カグマル
```

メソッドを追加して動きをつけよう

```
function Human(name, age) {
  this.name = name;
  this.age = age;

  this.greet = function() {
    console.log(`こんにちは、私は${this.name}です！`);
  };
}

const alice = new Human("アリス", 22);
alice.greet(); // こんにちは、私はアリスです！
```

コンストラクターを使わないパターンとの違い

```
const taro = {
  name: "太郎",
  age: 30
};

const hanako = {
  name: "花子",
  age: 25
};
```

よくあるミス

❌ new をつけ忘れる！

```
const taro = Human("太郎", 30); // ← new がない！
console.log(taro.name); // エラー or undefined になる
```

JavaScript：メソッドとプロパティの呼び出し & ドット演算子の授業

ドット演算子とは？

```
object.property
object.method()
```

この『.（ドット）』を ドット演算子（dot notation）と呼びます。

用語：・オブジェクトの中にある **「プロパティ」** を取り出す・オブジェクトの中にある **関数（メソッド）** を実行する

プロパティとは？（中に入ってる「情報」）

```
const apple = {
  color: "red",
  weight: 200
};

console.log(apple.color);  // red
console.log(apple.weight); // 200
```

メソッドとは？（中に入ってる「動き・命令」）

```
const apple = {
  color: "red",
  sayColor: function() {
    console.log(`私は${this.color}色のりんごです！`);
  }
};

apple.sayColor();  // 私は red色のりんごです！
```

ドット演算子の仕組み（図式）

```
オブジェクト名.プロパティ名
オブジェクト名.メソッド名()
```

例えるなら、オブジェクトは「キャラ」。ドット演算子は「アクセスする手」

よくあるパターン：オブジェクト＋ドット演算子

```
const teacher = {
  name: "アレンジオ",
  subject: "JavaScript",
  introduce: function() {
    console.log(`こんにちは！私は${this.name}です。担当は${this.subject}です。`);
  }
};

console.log(teacher.name);      // アレンジオ
console.log(teacher.subject);   // JavaScript
teacher.introduce();            // こんにちは！私はアレンジオです。担当はJavaScriptです。
```

JavaScriptの主なコンストラクターとメソッド一覧

メソッド一覧

```
String 関連メソッド

- .length → 文字数
- .toUpperCase() → 大文字に変換
- .toLowerCase() → 小文字に変換
- .indexOf("x") → 特定文字の位置を返す
- .includes("にち") → 含まれているか判定
- .slice(0, 3) → 一部を切り出す
- .replace("にち", "ばん") → 一部置換
```

```
Number 関連メソッド

- .toFixed(2) → 小数点以下を固定表示
- .toString() → 文字列に変換
- .valueOf() → 数値としての値を返す
```

```
Array 関連メソッド

-    .push("みかん") → 末尾に追加
-    .pop() → 末尾から削除
-    .shift() → 先頭から削除
-    .unshift("すいか") → 先頭に追加
-    .slice(1, 3) → 一部を取得
-    .splice(1, 1) → 一部を削除・変更
-    .join(", ") → 文字列として結合
-    .includes("バナナ")
-    .indexOf("バナナ")
-    .forEach(fn) → 繰り返し処理
-    .map(fn) → 配列変換
-    .filter(fn) → 条件に合う要素だけ残す
```

```
Object 関連メソッド

Object.keys(obj) → キー一覧
Object.values(obj) → 値一覧
Object.entries(obj) → [キー, 値] の配列
Object.assign({}, obj) → オブジェクトのコピー
hasOwnProperty("name") → プロパティがあるか？
```

```
Date 関連メソッド

- .toLocaleString() → 日本語日時表示
- .getFullYear() → 年を取得
- .getMonth() → 月（0〜11）
- .getDate() → 日にち
- .getHour() → 時間
- .getMinutes() → 分
- .getDay() → 曜日（0: 日曜）
```

```
Math 関連メソッド **

- Math.random() → 0〜1のランダム
- Math.floor(x) → 小数切り捨て
- Math.ceil(x) → 小数切り上げ
- Math.round(x) → 四捨五入
- Math.max(...numbers) → 最大値
- Math.min(...numbers) → 最小値
- Math.pow(x, y) → xのy乗
```

```
Map 関連メソッド

-    .set(key, value)
-    .get(key)
-    .has(key)
-    .delete(key)
-    .clear()
-    .size → 要素数
-    .forEach((v, k) => {})
```

```
** Set 関連メソッド**

- .add(value)
- .has(value)
- .delete(value)
- .clear()
- .size
- .forEach(v => {})
```

```
Promise 関連メソッド

- .then() → 成功時の処
```

コンストラクター　一覧

String コンストラクター

```
const str = new String("こんにちは");
```

Number コンストラクター

```
const num = new Number(42);
```

Boolean コンストラクター

```
const flag = new Boolean(true);
```

Array コンストラクター

```
const arr = new Array("りんご", "バナナ");
```

Object コンストラクター

```
const obj = new Object({ name: "太郎", age: 20 });
```

Date コンストラクター

```
const now = new Date();
```

RegExp コンストラクター

```
const pattern = new RegExp("abc");
```

Map コンストラクター

```
const myMap = new Map();
myMap.set("key", "value");
```

Set コンストラクター

```
const mySet = new Set([1, 2, 2, 3]);
```

Promise コンストラクター（非同期処理）

```
const p = new Promise((resolve, reject) => {
  resolve("成功！");
});
```

URL コンストラクター（URL解析）

```
const url = new URL("https://example.com?img=jp");
console.log(url.searchParams.get("img"); // "jp"
```

Error コンストラクター（エラー生成）

```
throw new Error("問題が発生しました！");
```

class + constructor（現代的な方法）

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log`${this.name}が鳴いた！`);
  }
}

const cat = new Animal("クマ");
cat.speak(); // クマが鳴いた！
```

Colab の有料サービス - 関数解除はこちら